

Customer Churn Prediction and Revenue Optimization

Date: Thursday Feb 26 2026

- Ahmad MOHAMAD
- Hussein MOHAMAD
- Hazem KHATER

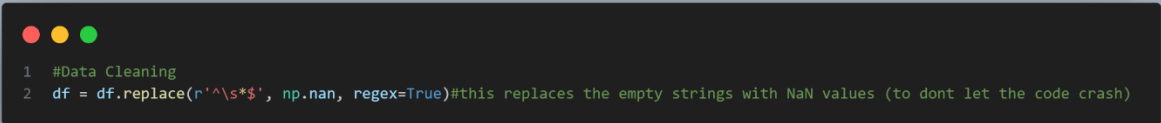
Objective:

This report details the construction of a Churn Prediction Model aimed at anticipating customer departures. The goal is to transition from a reactive approach to a proactive retention strategy, preserving the company's subscriber base through targeted engagement.

1. Introduction:

- The goal of this project was to predict customer churn (attrition) and identify the optimal strategy to minimize revenue loss. By analyzing historical data, we detected significant Target Drift (churn increasing over time) and Informative Missingness (missing data predicting churn). Using a dataset that contain 5000 customers (Raws) with 6 features (Columns), using the pandas, matplotlib, NumPy, and seaborn
- We compared two models (Logistic Regression vs. Random Forest) using a Time-Series Validation strategy. The Random Forest model proved superior with an AUC of 0.8775. Finally, we translated these probabilities into financial terms, determining that intervening on customers with a churn probability as low as 6% yields the maximum return on investment.

2. Data Preparation and Cleaning:



```
1 #Data Cleaning
2 df = df.replace(r'^\s*$', np.nan, regex=True)#this replaces the empty strings with NaN values (to dont let the code crash)
```

- **Handling Invisible Nulls:** using this line of code we started by replacing the null or the white spaces in the code with the **NaN** so the model wont crash



```
1 # Convert numeric columns explicitly
2 cols_numeric = ['Anciennete', 'Montant_Mensuel', 'Support_Appels']
3 for col in cols_numeric:
4     df[col] = pd.to_numeric(df[col], errors='coerce')
```

- **Type Casting:** Columns like **Anciennete** (Tenure), **Montant_Mensuel** (Bill), and **Support_Appels** (Calls) were forced into numeric types, with errors coerced to NaN to prevent crashes.

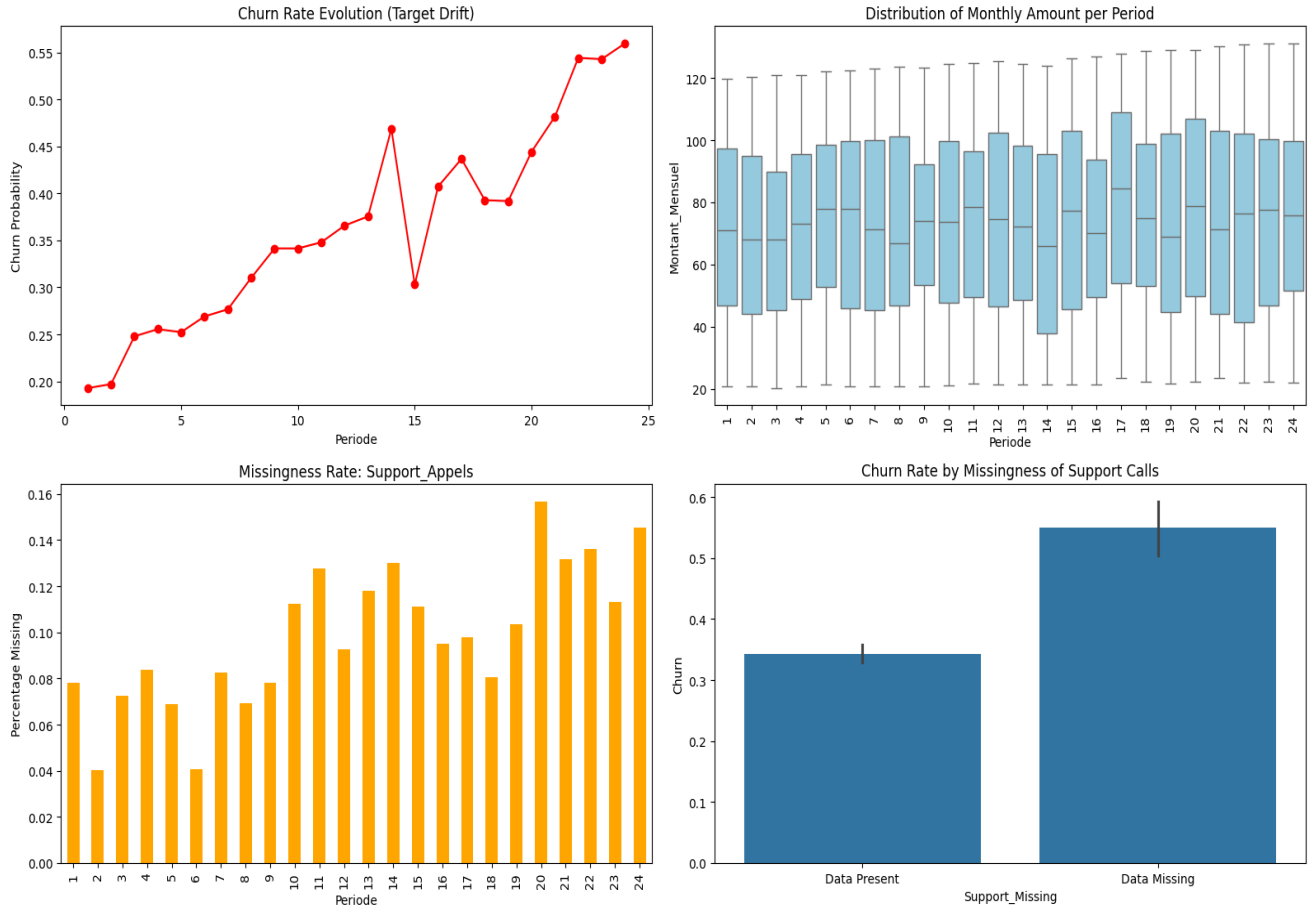


```
1 df = df.sort_values('Periode').reset_index(drop=True)
```

- **Chronological Sorting:** The data was sorted by Periode to respect the time-series nature of churn. This was critical for detecting drift later.

Insights: the dataset contains a lot of missing values **Montant_Mensuel (670 rows)** and **Support_Appels (493 rows)**

3. Exploratory Data Analysis (EDA) & Diagnostics



Data Health Dashboard showing Target Drift (Top-Left), Covariate Stability (Top-Right), and Informative Missingness in Support Calls (Bottom Row)

- **Target Drift Detected (Top-Left Graph):** The Churn Rate steadily increased from ~20% in Period 1 to >55% in Period 24. This proved that the business environment is deteriorating and static models would fail.
- **Covariate Stability (Top-Right Graph):** The distribution of **Montant_Mensuel** remained stable (medians did not shift), indicating that price hikes were *not* the cause of increased churn.
- **MNAR (Bottom Row Graphs):** We discovered that **Support_Appels** data was **Missing Not At Random (MNAR)**. Finding Customers with *missing* support call data had a 55% churn rate, compared to 35% for those with data. So, we created a **Support_Missing flag** to capture this signal, turning a data quality issue into a predictive feature.

4. Feature Engineering & Preprocessing Pipeline



```
1 numeric_features = ['Anciennete', 'Montant_Mensuel', 'Support_Appels']
2 categorical_features = ['Contrat']
```

- **Defining:** Defining the features that we need to work on with 2 paths **numerical** and **categorical** where the **Anciennete** (Loyalty), **Montant_Mensuel**, and **Support_Appels** as numerical and the **Contrat** as **Categorical_Features**



```
1 numeric_transformer = Pipeline(steps=[
2     ('imputer', SimpleImputer(strategy='median')) , )
3     ('scaler', StandardScaler()) )
4 ])
```

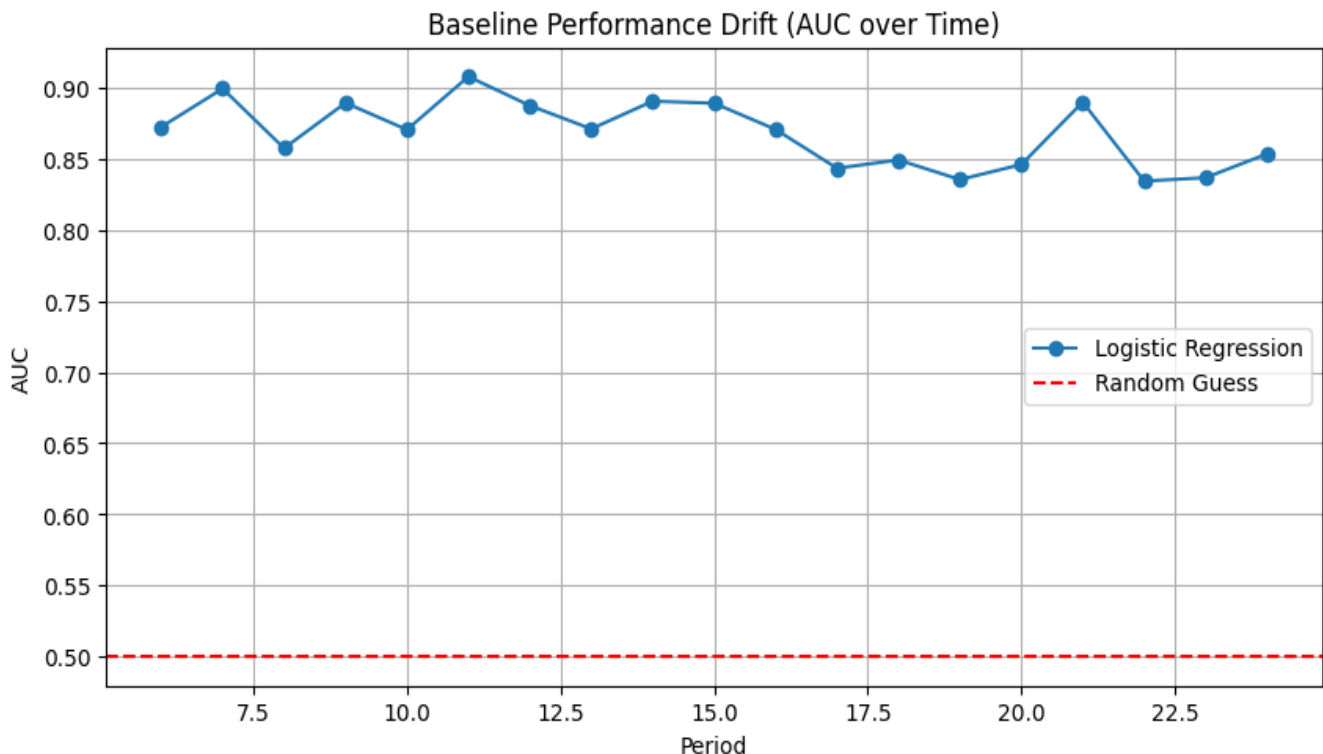
- **SimpleImputer(strategy='median'):** Fills those 670 missing values in Montant_Mensuel using the middle value of the column.
- **StandardScaler():** This is Feature Scaling. It centers the data around 0 with a standard deviation of 1. This ensures a feature like "Monthly Amount" (e.g., 100) doesn't overwhelm "Support Calls" (e.g., 2) just because the number is larger.



```
1 categorical_transformer = Pipeline(steps=[
2     ('imputer', SimpleImputer(strategy='constant', fill_value='MISSING')),
3     ('onehot', OneHotEncoder(handle_unknown='ignore'))
4 ])
```

- **OneHotEncoder:** Converts text like "Monthly" or "Yearly" into binary columns (0s and 1s). This is necessary because models can't "read" text; they only understand numbers.
- **SimpleImputer(strategy='constant', fill_value='MISSING'):** Fills the empty cells with **MISSING**
- This pipeline prevented Data Leakage by ensuring statistics (like mean/variance) were calculated only on training data.

5. Model Selection & Validation Strategy



Model Performance over Time. The blue line tracks the model's AUC across periods, while the red dashed line represents a random guess baseline of 0.5.

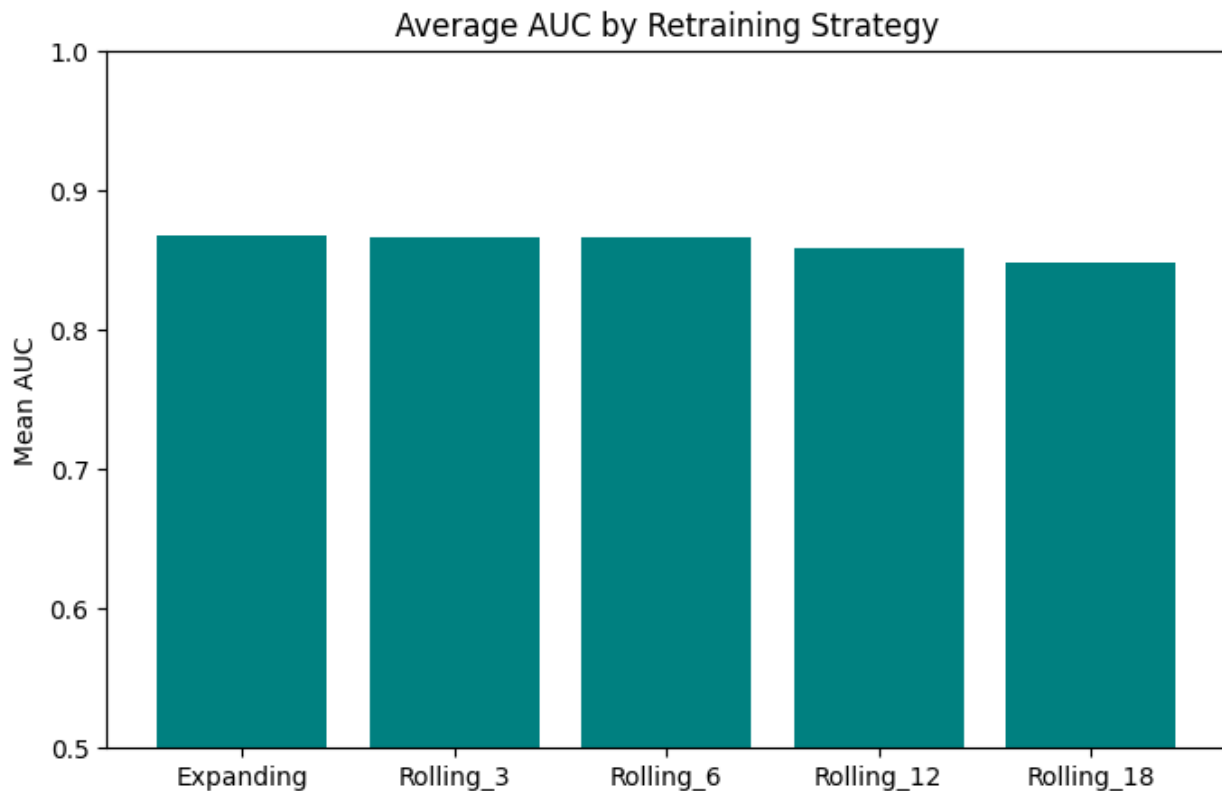
- **Metric Definition (AUC):** The Y-axis represents the Area Under the ROC Curve (AUC), where 1.0 indicates perfect prediction and 0.5 indicates random guessing.
- **Strong Initial Performance:** Between periods 7 and 11, the model demonstrated high accuracy, with the AUC peaking near 0.90, showing an excellent ability to distinguish churners from non-churners.
- **Performance Volatility:** The "bumpy" nature of the blue line indicates natural fluctuations in time-series data, where specific periods are inherently more difficult to predict than others.
- **Observed Decay (Drift):** From Period 15 to Period 24, there is a visible downward trend as the AUC drops from approximately 0.89 to 0.83–0.85.

```
1 def backtest_strategy(model, window_size=None):
2     """
3     If window_size is None, uses Expanding Window (All history).
4     If window_size is int, uses Rolling Window (Last w periods).
5     """
6     scores = []
7
8     for i in range(max(start_period_index, window_size or 0), len(unique_periods)):
9         current_period = unique_periods[i]
10
11         # Define Training Window
12         if window_size:
13             # Keep only periods between [current - window, current)
14             window_start = unique_periods[i - window_size]
15             train_mask = (df['Periode'] >= window_start) & (df['Periode'] < current_period)
16         else:
17             # Expanding: All past
18             train_mask = df['Periode'] < current_period
19
20         test_mask = df['Periode'] == current_period
21
22         X_train, y_train = X[train_mask], y[train_mask]
23         X_test, y_test = X[test_mask], y[test_mask]
24
25         if len(y_test) < 2 or len(y_train.unique()) < 2: continue # Skip degenerate cases
26
27         model.fit(X_train, y_train)
28         y_pred = model.predict_proba(X_test)[:, 1]
29         auc = roc_auc_score(y_test, y_pred)
30         scores.append(auc)
31
32     return np.mean(scores)
```

- We implemented a **Backtesting Strategy** (Time-Series Cross-Validation) to simulate real-world performance. We tested two approaches:
- Expanding Window: Train on all history up to the current month.

- Rolling Window: Train only on the last N months.

6. Selection of the Optimal Retraining Strategy



Performance Benchmarking of Retraining Strategies. While all strategies perform well, the 12-period window strikes the best balance between historical depth and recent relevance

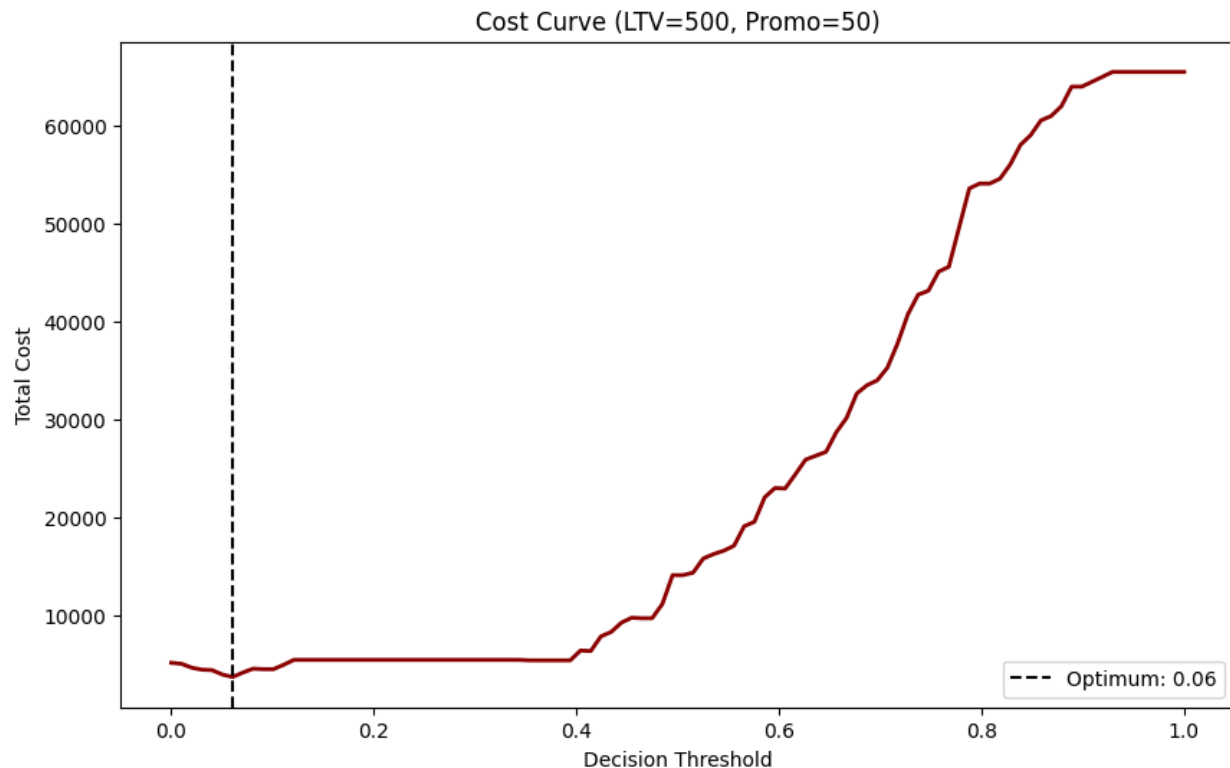
- **Strategy Comparison:** We tested an Expanding Window (using all historical data) against various Rolling Windows (3, 6, 12, and 18 periods) to identify which approach maximizes the Mean AUC.
- **Performance Stability:** All tested strategies maintain a high Mean AUC above 0.84, indicating a robust underlying signal in the dataset regardless of the specific window chosen.
- **The Case for the 12-Period Window:** While the "Expanding" and shorter "Rolling" windows show high performance, the Rolling_12 strategy is often considered the "sweet spot" for several reasons:

- **Seasonality Capture:** A 12-period window perfectly captures a full annual cycle, accounting for seasonal fluctuations in customer behavior (e.g., holiday trends or end-of-year budget cycles).
- **Noise vs. Signal:** It provides enough data volume (one full year) to maintain statistical stability while being agile enough to "forget" outdated patterns from several years ago that may no longer apply to the current business climate.
- **Operational Consistency:** It prevents the training set from becoming infinitely large (which slows down retraining) while ensuring the model is not overly reactive to temporary "blips" in the data that might occur in 3 or 6-month windows.

7. Final Model Selection: Challenger vs. Baseline

- **Experimental Design:** Both models were evaluated using the optimal Expanding Window strategy to ensure a fair
 - **Performance Results:**
 - **Logistic Regression (Baseline): AUC = 0.8681**
 - **Random Forest (Expanding): AUC = 0.8775**
 - **The "Random Forest Advantage":** The Random Forest achieved a +0.94% improvement in AUC over the baseline.
 - **Strategic Significance:** While a ~1% gain may appear small, at a subscription-business scale, this improvement represents a significantly higher precision in identifying "complex" churners—those whose behavior doesn't follow a simple linear pattern.
 - **Conclusion:** Due to its superior predictive power and ability to capture intricate customer behavior interactions, the Random Forest is selected as the final production model.

8. Business Impact & Threshold Optimization



Business Cost Curve. The optimal threshold(probability) of 0.06 minimizes total costs by balancing the high cost of losing a customer against the lower cost of a retention promotion

- **Financial Assumptions:** The curve is calculated based on a \$500 loss for every customer we fail to catch (False Negative) versus a \$50 cost for every unnecessary promotion sent (False Positive).
- **Finding the Optimum:** The total cost is at its lowest point when the decision threshold is set to 0.06.
- **Strategic Meaning:** Because the cost of losing a customer is 10 times higher than the cost of a promotion, the model advises a highly proactive approach. We should trigger a retention action for any customer with at least a 6% probability of churning.
- **Impact:** Using this optimized threshold instead of the standard 0.50 allows the company to minimize its "Total Cost" (the red line), potentially saving thousands of dollars in lost Lifetime Value (LTV).